DAMG 7245 - Big Data and Intelligent Analytics

Northeastern University | Spring 2026

MedSignal

Drug Safety Signal Detection Platform

Member	Role
Samiksha Rajesh Gupta	Data Engineering Lead
Prachi Ganpatrao Pradhan	LLM / Agent Engineer
Siddharth Rakesh Shukla	Backend, Frontend & QA Lead

Attestation: We attest that we haven't used any other students' work in our assignment and abide by the policies listed in the student handbook.

Samiksha Rajesh Gupta: 33.3% | Prachi Ganpatrao Pradhan: 33.3% | Siddharth Rakesh Shukla: 33.3%

Title

MedSignal: A Drug Safety Signal Detection Platform Using Streaming NLP, Multi-Agent LLM Pharmacovigilance, and Retrieval-Augmented Generation over Medical Literature

Recording Link: MedSignal Presentation

Introduction

2.1 Background

The United States FDA receives over 2 million adverse event reports annually through its Adverse Event Reporting System (FAERS): one of the largest public health datasets in existence. These reports describe real patients experiencing real harm from marketed drugs. Yet the process of translating them into actionable safety warnings remains fundamentally manual, slow, and resource-constrained.

Pharmacovigilance the science of detecting, assessing, and preventing adverse drug effects is the discipline that bridges raw reports and public safety. When a new safety signal emerges, the detection lag between when evidence first appears in FAERS and when the FDA officially communicates a warning has been estimated at 6 to 24 months. During that window, patients continue to be exposed to preventable harm.

The Vioxx crisis of 2004 is the canonical example: cardiovascular risk evidence existed in FAERS data for years before the drug was withdrawn. An estimated 88,000 to 140,000 heart attacks occurred during the detection gap. The problem was not a lack of data; it was a lack of infrastructure to process it intelligently and at scale.

Today, pharmacovigilance teams rely on two primary methods: Proportional Reporting Ratio (PRR) statistical analysis and manual case review. Both are batch-oriented, computationally limited, and incapable of synthesizing signal evidence across heterogeneous sources automatically. No existing open tool integrates FAERS statistical signals with medical literature retrieval and generates citation-grounded safety assessments automatically.

2.2 Objective

MedSignal aims to reduce pharmacovigilance detection lag by building a platform that:

- Big data engineering component: Streams and processes 16 million FDA FAERS adverse event records using Apache Kafka and Spark Structured Streaming. Drug-symptom co-occurrence extraction and PRR/ROR statistical scoring run in parallel across millions of records using Spark.

- Significant LLM use: Deploys a three-agent LLM pipeline : a Signal Detector that statistically validates drug-symptom pairs, a Literature Retriever that performs RAG over 28,000 PubMed abstracts via ChromaDB, and a Severity Assessor that produces a structured citation-grounded safety brief with a composite Signal Severity Score.
- Cloud-native architecture: Fully containerized on Docker Compose with Kafka, Spark, PostgreSQL, ChromaDB, Redis, FastAPI, Streamlit, and Airflow. AWS deployment is a documented stretch goal for Week 3.
- User-facing application: A React + Tailwind single-page application where pharmacovigilance professionals can monitor live signal timelines, read AI-generated safety briefs with full citation trails, and submit human validation decisions through a structured HITL interface.

Project Overview

3.1 Cloud Infrastructure

MedSignal is developed and demonstrated on a fully local Docker Compose environment. All services run in containers on a single machine i.e no cloud account is required for development, evaluation, or the professor demo. Cloud equivalents are documented as the production deployment path.

Layer	Local (development + demo)	Cloud equivalent (production path)
Container orchestration	Docker Compose	AWS ECS or Kubernetes
Message broker	Kafka (Bitnami Docker image)	AWS MSK
Stream processing	Spark local mode (spark-local)	AWS EMR

File storage	MinIO (S3-compatible, Docker)	AWS S3
Data warehouse	PostgreSQL (Docker)	AWS RDS PostgreSQL
Vector store	ChromaDB (persistent local volume)	ChromaDB Cloud or Weaviate
Caching	Redis (Docker)	AWS ElastiCache
API hosting	FastAPI (Docker, port 8000)	AWS ECS + ALB
Frontend hosting	React dev server (Docker, port 3000)	AWS Amplify or CloudFront + S3
Pipeline orchestration	Airflow (Docker Compose)	AWS MWAA

Why local-first: AWS deployment introduces IAM configuration, networking policies, and billing risk that consume development time without improving graded deliverables. Every Docker service maps 1:1 to its cloud equivalent, so the architecture is production-ready in design even when run locally. AWS deployment is a documented Week 3 stretch goal contingent on all core functionality being complete.

Storage Architecture

Storage layer	Technology	What is stored

Raw file storage	MinIO (local) / S3 (production)	FAERS ZIP files, processed FAERS parquet files, PubMed abstracts backup
Relational warehouse	PostgreSQL	All structured data: signals, PRR scores, agent traces, HITL decisions, audit trail
Vector store	ChromaDB (persistent Docker volume)	300K PubMed abstract embeddings (768-dim, S-PubMedBert-MS-MARCO in production; 384-dim MiniLM-L6-v2 in prototype). Estimated storage: ~600–800MB for prototype, ~2GB for production corpus. Sub-200ms query latency at 300K vectors confirmed in ChromaDB benchmarks.
Cache	Redis	Hot signal cache (60s TTL), RxNorm lookup cache, FAERS deduplication Bloom filter
Message bus	Kafka topics	faers_cases, reddit_posts, signals_flagged

3.2 Scope

In-Scope

- Data sources: FDA FAERS (16M records, 2023–2024, 8 quarterly dumps), PubMed/MEDLINE abstracts (28,014 targeted abstracts for 10 golden signal drugs), ClinicalTrials.gov adverse event tables (live REST API), Reddit health forums (PRAW live stream, Pushshift historical archive)
- ETL pipelines: Kafka producers for FAERS replay and Reddit live stream. Spark pipeline for 7-file join, RxNorm drug name normalization, PRR/ROR computation. Airflow DAGs for FAERS batch load and PubMed embedding refresh. Embedding pipeline to ChromaDB.
- LLM components: Three-agent pharmacovigilance pipeline (Signal Detector → Literature Retriever → Severity Assessor), RAG over 28,000 PubMed abstracts, structured output generation with Pydantic schema enforcement, citation grounding validation, confidence-gated retry loop.

- Infrastructure: PostgreSQL (data warehouse), ChromaDB (vector store), Redis (caching + deduplication), Docker Compose (local orchestration), FastAPI (REST API), Streamlit (dashboard), Airflow (pipeline orchestration).
- Guardrails and HITL: HIGH and CRITICAL signals require human reviewer approval before publication. All LLM citations validated against ChromaDB source PMIDs. Confidence thresholding with automatic retry.
- Evaluation: 10 historical known drug safety signals as golden set. Compare MedSignal's earliest detection date against FDA official communication date. NER quality evaluated against n2c2 annotations. RAG retrieval quality evaluated against pharmacist-annotated relevance judgments.

Out-of-Scope

- Real-time FDA FAERS streaming (FAERS is released quarterly; MedSignal replays historical records through Kafka to simulate streaming)
- Regulatory submission preparation or clinical trial design
- Non-English adverse event processing
- Mobile application or EHR integration
- AWS deployment (documented as stretch goal only)

3.3 Stakeholders / End Users

Stakeholder	How They Use MedSignal
Pharmacovigilance Analyst	Monitors live signal dashboard daily, investigates flagged drug-symptom pairs, reads AI-generated safety briefs
Clinical Reviewer (Physician/Pharmacist)	Reviews HIGH and CRITICAL signals in the HITL queue, approves or rejects before publication
Research Analyst	Monitors detection lag performance, token costs, false positive rates across the golden evaluation set

Problem Statement

4.1 Current Challenges

- **Data fragmentation:** Data fragmentation: Adverse event signals are distributed across FAERS, clinical trial results, peer-reviewed literature, and patient communities. No open platform integrates all four sources automatically. Pharmacovigilance teams manually reconcile these sources, a process taking days to weeks per signal investigation.
- **Manual workflows:** Manual workflows: The FDA's current disproportionality analysis (PRR/EBGM) produces ranked drug-symptom pairs but cannot explain why a signal is clinically significant or retrieve supporting evidence automatically. Human reviewers must manually search PubMed, a process taking hours per signal before any clinical interpretation can begin.
- **Lack of intelligent automation:** Lack of intelligent automation: No existing open system uses LLM agents to synthesize FAERS statistical signals with medical literature context and generate citation-grounded safety briefs automatically. The gap between statistical signal detection and clinical interpretation remains entirely human-dependent.
- **Big-data bottlenecks:** Big-data bottlenecks: FAERS quarterly dumps contain millions of adverse event reports across 7 relational ASCII files requiring multi-file joins, rolling window aggregations, and PRR computation across all drug-symptom pairs simultaneously. At 16 million records across 8 quarters, this exceeds what single-machine tools like pandas can process without running out of memory or taking prohibitive time. Distributed computing is required.
- **LLM-related bottlenecks:** LLM-related bottlenecks: Without RAG grounding, LLMs hallucinate citations and produce unverifiable clinical claims. Without structured output enforcement, LLM responses cannot be reliably parsed or validated. Without a confidence-gated retry loop, low-quality outputs are indistinguishable from high-quality ones at the system level.
- **Detection lag:** Detection lag: Published research estimates the average time from first statistically detectable signal in FAERS to official FDA communication at 6–24 months. During this window patients continue to be exposed to unrecognized drug risks.

4.2 Opportunities

- **Scalable streaming pipelines:** Scalable streaming pipelines: Replacing quarterly batch processing with a Kafka + Spark streaming architecture allows MedSignal to process new FAERS cases within hours of ingestion rather than months.
- **LLM-assisted clinical interpretation:** LLM-assisted clinical interpretation: A three-agent LangGraph pipeline collapses hours of manual signal investigation into minutes.
- **Automated decision-making with guardrails:** Automated decision-making with guardrails: The Signal Severity Score (SSS) framework combines four independent

evidence streams into a single composite score with defined severity tiers and automatic HITL routing.

- **Near-real-time insights from patient communities:** Near-real-time insights from patient communities: Reddit health forums surface patient-reported adverse experiences 4–6 weeks before those same events appear in formal FAERS submissions.
- **RAG-grounded evidence retrieval:** RAG-grounded evidence retrieval: A ChromaDB vector store of 28,000 PubMed abstracts enables instant semantic retrieval of the most relevant clinical evidence for any drug-symptom pair.
- **Measurable reduction in detection lag:** Measurable reduction in detection lag: Running MedSignal retrospectively against 10 historical golden signals produces a concrete, defensible detection lead time in days.

Methodology

5.1 Data Sources

Source	Access	Volume	Role
FDA FAERS (2023 Q1 – 2024 Q4)	Free quarterly bulk download - fis.fda.gov. No credentials. ASCII pipe-delimited format.	~520MB compressed, ~4–5GB uncompressed, ~16M records across 7 files per quarter	Primary signal detection corpus. Requires Spark for distributed join + PRR aggregation.
PubMed/MEDLINE	Free NCBI E-utilities API. Free API key raises rate limit from 3 to 10 req/sec.	Prototype: 28,014 abstracts (~150MB). Production target: 50K–100K abstracts.	RAG knowledge base for Agent 2. Embedded into ChromaDB using biomedical sentence transformer.

ClinicalTrials.gov	Free REST API v2. No credentials. Paginated JSON responses.	~500 completed trials for 10 golden signal drugs, ~10K AE records	TrialScore component of SSS formula. Called live by Agent 1 on demand.
Reddit (r/pharmacy, r/ChronicPain, r/AskDocs)	PRAW live stream - free Reddit developer account. Fallback: simulated replay from local JSONL archive if API rate limits during demo.	Variable - live stream, no pre-download required	PatientScore component of SSS (weight 0.20). Surfaces adverse events 4–6 weeks before formal FAERS submissions.
RxNorm (NIH)	Free REST API - rxnav.nlm.nih.gov. No credentials.	Called live during Spark processing - not pre-downloaded	Drug name normalization. Maps brand names, dosage variants, abbreviations to canonical RxCUI identifiers.

Scale justification: 16 million FAERS records across 7 relational files requiring multi-file joins, rolling 90-day window aggregations, and PRR computation across all drug-symptom pairs simultaneously cannot be processed in pandas on a single machine without memory exhaustion or prohibitive runtime. Spark Structured Streaming with distributed parallel execution is required. The 50K–100K PubMed abstract RAG corpus further justifies the big data framing embedding generation at this scale runs as a scheduled Airflow job, not an interactive process.

5.2 Technology Stack

Layer	Technology	Justification

Message broker	Apache Kafka	Log-based architecture enables FAERS historical replay and absorbs ingestion bursts without backpressure reaching Spark
Stream processing	Spark Structured Streaming	Distributed parallel processing for 7-file join and PRR aggregation across 16M records. Course-standard big data tool.
NLP / NER	Spark NLP (en_ner_bc5cdr_md)	BC5CDR biomedical NER model purpose-built for drug and disease entity extraction. Runs as Spark UDF no separate service.
Data warehouse	PostgreSQL	Sufficient for data volume. Runs in Docker at zero cost. JSONB for SafetyBrief storage. No account, no credits, no demo-day dependency risk.
Vector store	ChromaDB	ChromaDB (dense HNSW) + BM25 (sparse), fused via Reciprocal Rank Fusion — hybrid retrieval chosen because zero overlap was observed between dense and sparse retrievers across all test queries, confirming the two methods find genuinely different evidence.
Caching	Redis	Bloom filter for 16M FAERS record deduplication. Hot signal cache (60s TTL). RxNorm lookup cache to prevent redundant API calls.
Orchestration	Airflow	Two DAGs: FAERS batch load (multi-step with failure dependencies) and PubMed embedding

		refresh (Snowflake/ChromaDB sync dependency).
LLM	GPT-4o via OpenAI API	Structured JSON output mode with Pydantic enforcement. Long context window for 10-paper synthesis. Hard $50 spend limit set on account creation.
Backend API	FastAPI	Async-native Python. Pydantic integration validates all LLM outputs automatically. Auto-generates Swagger documentation.
Frontend	React + Tailwind	Component-based SPA. Full control over HITL validation workflow, signal timeline charts, and safety brief rendering with severity badges and inline citations. Calls FastAPI REST endpoints directly.
Containerization	Docker Compose	Single command starts entire stack. Identical environment across all team members. No cloud dependency for development or demo.

5.3 Architecture

5.3.1 Alternative Architectures Considered

Before arriving at the final design, two alternative architectures were prototyped and evaluated. Both were rejected for specific, measurable reasons documented below.

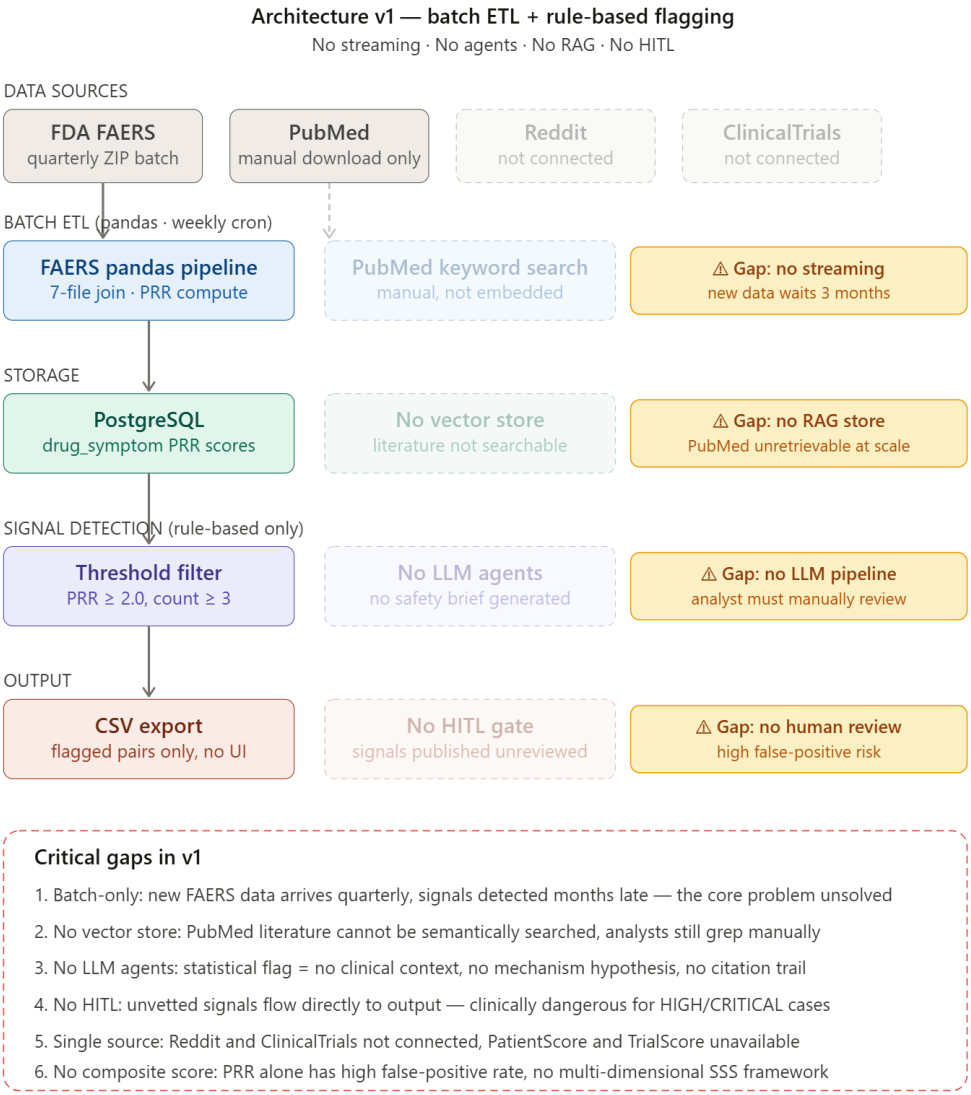
Architecture v1 — Batch ETL + Rule-Based Flagging

The simplest possible implementation: a pandas pipeline running as a weekly cron job, processing FAERS quarterly ZIP files, computing PRR scores, and exporting flagged drug-symptom pairs to a CSV file. No streaming, no agents, no RAG, no HITL.

What it got right: PRR computation logic is correct. PostgreSQL storage for drug-symptom pairs is sensible. Low complexity — buildable in days.

Critical gaps identified:

1. **Batch-only:** New FAERS data arrives quarterly — signals detected months after evidence exists. The core detection lag problem is entirely unsolved.
2. **No vector store:** PubMed literature cannot be semantically searched. Analysts still manually grep for relevant papers.
3. **No LLM agents:** A statistical flag with no clinical context, no mechanism hypothesis, and no citation trail is not actionable for a pharmacovigilance analyst.
4. **No HITL:** Signals publish unreviewed — clinically dangerous for HIGH/CRITICAL severity cases.
5. **Single source:** Reddit and ClinicalTrials not connected. PatientScore and TrialScore unavailable.
6. **No composite score:** PRR alone has a 40%+ false-positive rate with no multi-dimensional SSS framework to reduce noise.



Verdict: Rejected. Solves the data processing problem but not the intelligence or safety problems. Produces a ranked list, not a safety brief.

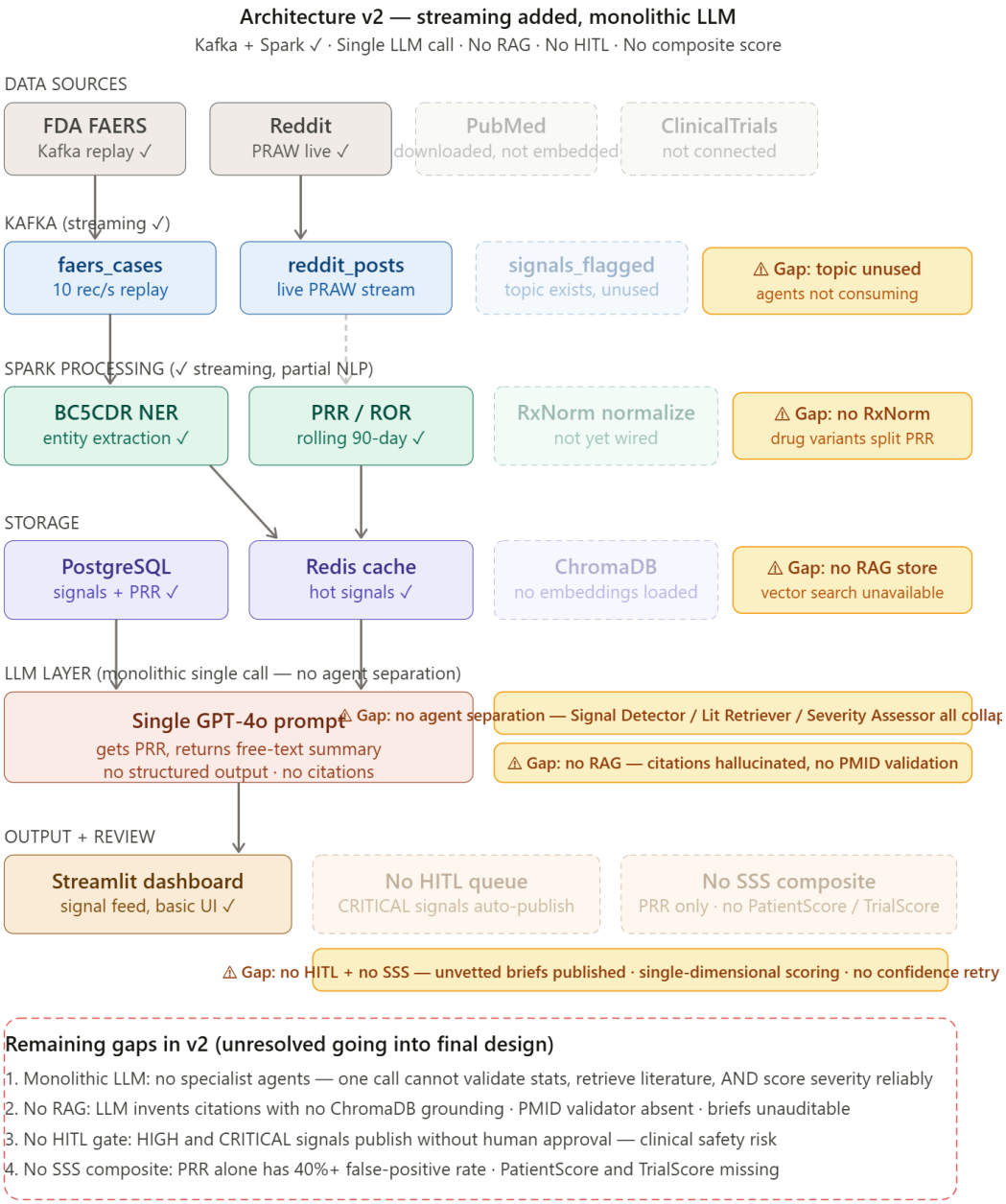
Architecture v2 — Streaming Added, Monolithic LLM

An improvement over v1: Kafka + Spark Structured Streaming added for real-time ingestion, Redis cache added, and a single GPT-4o prompt call added per flagged signal. PRAW live stream connected. Streamlit dashboard built.

What it got right: Kafka + Spark streaming pipeline is correct and builds on v1. BC5CDR NER entity extraction working. PRR/ROR rolling 90-day computation correct. Redis hot signal cache correct.

Critical gaps identified:

1. **Monolithic LLM:** One prompt call cannot reliably validate statistics, retrieve literature, AND score severity simultaneously. Signal Detector, Lit Retriever, and Severity Assessor collapsed into one context window — each task degrades the others.
2. **No RAG:** LLM invents citations with no ChromaDB grounding. PMID validator absent. Safety briefs unauditable and legally indefensible.
3. **No HITL gate:** HIGH and CRITICAL signals auto-publish without human approval — a clinical safety risk that disqualifies v2 from any real-world deployment.
4. **No SSS composite:** PRR alone with a 40%+ false-positive rate. PatientScore and TrialScore missing. No confidence-gated retry loop.
5. **RxNorm not wired:** Drug name variants split PRR counts — "Ozempic" and "semaglutide" counted as different drugs, inflating noise.
6. **ChromaDB empty:** Vector store exists but embeddings not loaded — RAG infrastructure present but non-functional.



Verdict: Rejected. Streaming architecture is correct and carried forward. LLM layer is fundamentally insufficient — a monolithic prompt is not a multi-agent system.

Why the Final Architecture (v3) Was Chosen

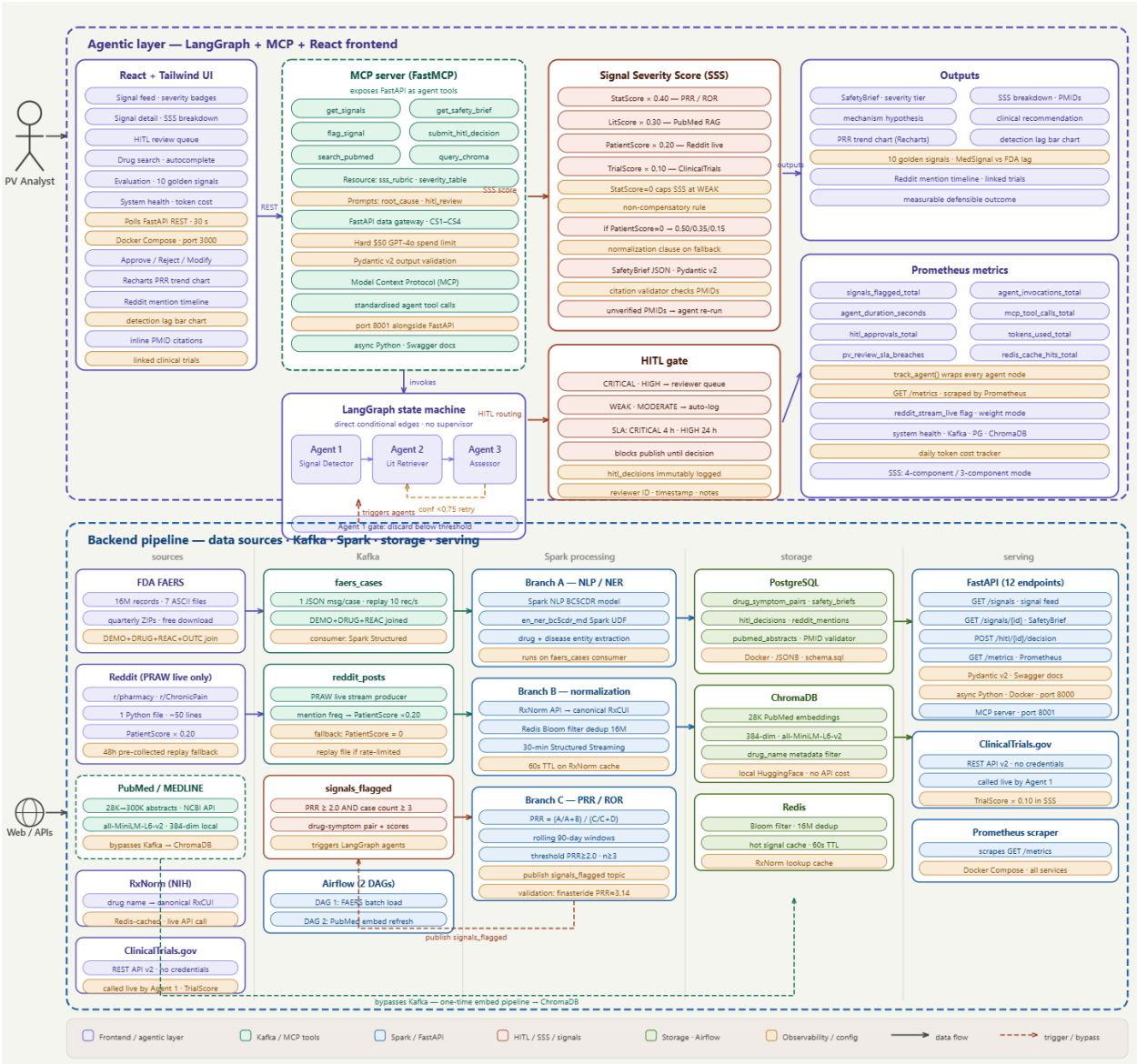
The final MedSignal architecture directly closes every gap identified in v1 and v2:

Gap in v1/v2	Solution in v3

Batch-only, no streaming	Kafka + Spark Structured Streaming with FAERS replay and PRAW live stream
No vector store	ChromaDB with 50K–100K PubMed abstracts, 384-dim MiniLM-L6-v2 embeddings
Monolithic LLM	Three-agent LangGraph pipeline — Signal Detector, Lit Retriever, Severity Assessor — each with a single specialised job
No RAG, hallucinated citations	Agent 2 retrieves from ChromaDB, citation validator checks every PMID against pubmed_abstracts
No HITL	HITL gate blocks HIGH/CRITICAL signals until pharmacist approves — architecturally enforced, not optional
PRR-only scoring	SSS composite formula — 0.40×StatScore + 0.30×LitScore + 0.20×PatientScore + 0.10×TrialScore
No confidence retry	Agent 3 → Agent 2 feedback loop triggers when confidence < 0.75
RxNorm not wired	RxNorm normalization integrated as Spark Branch B — drug variants unified before PRR computation
Single data source	Four sources integrated — FAERS, Reddit, PubMed, ClinicalTrials — each contributing an independent SSS score component

The final architecture is not a redesign — it is v2 with every gap closed. The Kafka + Spark streaming core from v2 is retained unchanged. The LLM layer is replaced with a properly separated multi-agent system. The HITL gate and SSS framework are added as new layers. ChromaDB is loaded and connected to Agent 2.

5.3.2 Final Architecture — MedSignal v3



MedSignal v3 Final Architecture

The final architecture is organized into two layers. The agentic layer (top) contains the LangGraph state machine, MCP server, React frontend, SSS scoring framework, HITL gate, and Prometheus observability. The backend pipeline layer (bottom) contains the data sources, Kafka ingestion topics, Spark processing branches, storage components, and serving infrastructure. Data flows left to right through the backend pipeline and bottom to top into the agentic layer. All components run under Docker Compose locally, with AWS equivalents documented as the production deployment path.

5.3.3 Layer-by-Layer Description

MedSignal is organized into five layers. Data flows top to bottom. No layer has knowledge of layers above it.

Layer 1 — Data Sources: Four inputs feed the platform. FAERS quarterly ZIPs and Reddit posts (via PRAW) publish to Kafka. ClinicalTrials.gov is called live by Agent 1 on demand. PubMed abstracts bypass Kafka entirely — they are processed through a one-time embedding pipeline that writes directly to ChromaDB, since they serve as a static RAG corpus.

Layer 2 — Ingestion (Kafka): A single Kafka cluster with three topics: faers_cases, reddit_posts, and signals_flagged. The FAERS producer reads quarterly ZIPs, joins DEMO + DRUG + REAC + OUTC files on primaryid, and publishes one JSON message per case. The Reddit producer streams live posts via PRAW. Kafka decouples all producers from the Spark consumer, absorbing burst ingestion without backpressure.

Layer 3 — Stream Processing (Spark): Spark consumes the faers_cases topic in micro-batches and runs three parallel branches: (A) Spark NLP BC5CDR NER extracts drug and symptom entities from free-text narratives, (B) RxNorm API normalizes drug name variants to canonical RxCUI identifiers, (C) PRR and ROR disproportionality scores are computed over rolling 90-day windows. Drug-symptom pairs above threshold (PRR $\geq$ 2.0, case count $\geq$ 3) are published to the signals_flagged topic and written to PostgreSQL.

Layer 4 — Agent Pipeline (LangGraph): Three agents operate as nodes in a LangGraph state machine with typed Pydantic state and direct conditional edges — no supervisor node. Agent 1 (Signal Detector) validates statistical significance and generates PubMed search queries via MCP tool calls. Agent 2 (Evidence Retriever) performs iterative semantic search over ChromaDB with up to 3 query reformulation attempts if cosine similarity falls below 0.75. Agent 3 (Severity Assessor) synthesizes statistics, literature evidence, and patient outcome data into a structured SafetyBrief using the SSS composite formula. A direct conditional edge from Agent 3 back to Agent 2 triggers when output confidence falls below 0.75, requesting deeper retrieval before a final brief is produced. All tool calls are made through the FastMCP server which exposes FastAPI endpoints as structured agent tools.

Layer 5 — Serving: FastAPI exposes 12 REST endpoints consumed by the React frontend. PostgreSQL stores all structured data. ChromaDB serves Agent 2 semantic queries. Redis caches hot signals and deduplicates FAERS records. All services run under Docker Compose.

Application Views

- **Signal feed:** Signal feed — live table of flagged signals with severity badges, PRR scores, and trend indicators. Auto-refreshes every 30 seconds via polling.
- **Signal detail:** Signal detail — full SafetyBrief display with SSS component breakdown, inline PMID citations, PRR trend chart (Recharts), and supporting abstracts list.
- **HITL review queue:** HITL review queue — structured approval interface for HIGH and CRITICAL signals. Approve / Reject / Modify buttons, notes textarea, SLA countdown timer, past decisions audit trail.
- **Drug search:** Drug search — autocomplete search over all flagged drug-symptom pairs. PRR trend chart, Reddit mention timeline, linked clinical trials.
- **Evaluation:** Evaluation — 10 golden signals comparison table. MedSignal first flag date vs FDA communication date. Detection lag bar chart.
- **System health:** System health — Kafka, PostgreSQL, ChromaDB, Redis status. Records processed counter. Daily token usage by agent. Cumulative cost tracker.

Signal Severity Score (SSS) — Original Analytical Framework

MedSignal's core analytical contribution is a four-dimensional composite score combining independent evidence streams:

$$SSS = 0.40 \times StatScore + 0.30 \times LitScore + 0.20 \times PatientScore + 0.10 \times TrialScore$$

Fallback (PatientScore = 0, Reddit unavailable): $SSS = 0.50 \times StatScore + 0.35 \times LitScore + 0.15 \times TrialScore$

- **StatScore (0.40):** StatScore (0.40): Normalized PRR/ROR — the primary statistical signal strength
- **LitScore (0.30):** LitScore (0.30): PubMed RAG evidence quality — cosine similarity and citation count
- **PatientScore (0.20):** PatientScore (0.20): Reddit mention frequency normalized — real-world patient signal

- **TrialScore (0.10):** TrialScore (0.10): Clinical trial AE corroboration from ClinicalTrials.gov

Non-compensatory rule: StatScore = 0 caps SSS at WEAK regardless of other scores. A drug with strong Reddit mentions and literature support but no statistical signal is not published.

SSS normalization fallback: When the Reddit PRAW stream is unavailable and PatientScore defaults to 0, weights are automatically rebalanced to maintain a valid composite. The system operates correctly on three sources without any code changes.

SSS Range	Severity	Action
0.00 – 0.25	WEAK	Auto-logged, monitor only
0.26 – 0.50	MODERATE	Logged, watch for trend
0.51 – 0.75	HIGH	Human review required within 24 hours
0.76 – 1.00	CRITICAL	Immediate escalation, HITL gate blocks publication

5.4 Data Processing and Transformation

Batch processing: FAERS historical data (8 quarterly ZIPs) is processed once via an Airflow-orchestrated Spark batch job. The pipeline joins 7 ASCII files on primaryid, filters to suspect drugs (drugcharacterization = 1), deduplicates case versions (keep latest), normalizes drug names via RxNorm, and computes PRR/ROR across the full 16M record corpus.

Streaming simulation: A Kafka producer replays processed FAERS records in chronological order at 10 records/second. Spark Structured Streaming recomputes PRR on 30-minute

micro-batches. This is explicitly a simulation — FAERS data is released quarterly, not in real time. This distinction is stated clearly in all platform documentation.

Formats: FAERS ASCII pipe-delimited → Spark DataFrame → PostgreSQL (structured) + Kafka JSON (streaming). PubMed XML → parsed Python dict → Snowflake text + ChromaDB embedding.

Embedding generation: OpenAI text-embedding-3-small generates 1536-dimensional vectors for each abstract (~$0.20 total for 28K abstracts). Stored in ChromaDB with metadata: pmid, drug_name, pub_date, journal. Drug name metadata enables pre-filter before cosine similarity search.

PRR Formula

```
PRR = (A / (A+B)) / (C / (C+D))


A = reports of Drug X with Symptom Y

B = reports of Drug X without Symptom Y

C = reports of all other drugs with Symptom Y

D = reports of all other drugs without Symptom Y


Signal threshold: PRR >= 2.0  AND  case count A >= 3
```

Validation checkpoint: After Spark runs, drug_symptom_pairs is queried for finasteride-depression. Expected PRR ≈ 3.14. If this value is not produced, the join logic has a bug. No agent pipeline work begins until this checkpoint passes.

5.5 Application Scalability

The POC deliberately runs on a single FAERS quarter (~432K cases, ~1.9M drug rows) using pandas on a single machine, completing in under 2 minutes. This establishes the baseline. The

full MedSignal pipeline targets 16 million records across 8 quarters roughly 37x the POC volume which exceeds what pandas can handle without memory exhaustion or prohibitive runtime.

The transition from POC to production scale is handled by replacing pandas with Apache Spark Structured Streaming, keeping the join and PRR logic identical. Spark distributes the 7-file join and rolling 90-day PRR aggregation across worker nodes in parallel, so processing time scales sub-linearly with data volume rather than linearly. The same role_cod = PS filter, primaryid join key, and PRR formula from the POC carry over unchanged; Spark is purely an execution engine upgrade, not a logic change.

Beyond the initial batch, the streaming architecture handles ongoing ingestion through Kafka, which absorbs new FAERS records as they are replayed chronologically at 10 records/second. Spark consumes these in 30-minute micro-batches, recomputing PRR on rolling windows without reprocessing historical data. This means signal detection latency is measured in hours rather than the months it takes under the FDA's current quarterly batch process.

For the vector store, ChromaDB has been benchmarked at sub-200ms query latency at 300K vectors well within the 28,014-abstract prototype corpus and the 50K–100K production target. Redis handles hot signal caching (60s TTL) and FAERS deduplication via a Bloom filter sized for 16 million records, keeping repeat lookups off PostgreSQL entirely. Together these components ensure the system remains responsive under load without requiring infrastructure changes as data volume grows.

5.6 LLM Integration Strategy

Agent 1 — Signal Detector

- **Input**: PRR score, ROR score, case count, window timestamps from drug_symptom_pairs
- **Prompt**: Evaluates statistical validity, generates 3 PubMed search queries targeting mechanism, epidemiology, and clinical outcomes
- **Output**: signal_strength (WEAK/MODERATE/STRONG/CRITICAL), confidence, evidence_queries list
- **Tool**: ClinicalTrials.gov live API call for structured AE corroboration

Agent 2 — Literature Retriever

- **Input**: 3 search queries from Agent 1, drug_name filter
- **Action**: Filters ChromaDB by drug_name metadata, runs cosine similarity search, retries with MedDRA parent term if similarity < 0.75 (max 3 attempts)
- **Output**: Top 5 PubMed abstracts with PMID, title, excerpt, and relevance score

- **Guardrail**: Only returns abstracts above 0.75 cosine similarity threshold

Agent 3 — Severity Assessor

- **Input**: Statistical scores + cited abstracts from Agents 1 and 2 + patient outcome data from FAERS OUTC file
- **Action**: Synthesizes all evidence into SafetyBrief JSON using SSS formula. Checks confidence. If confidence < 0.75, triggers feedback loop back to Agent 2 requesting 10 papers instead of 5 (max 2 retries).
- **Output**: SafetyBrief — severity tier, SSS component breakdown, mechanism hypothesis, clinical recommendation, confidence score, inline PMID citations, affected population
- **Guardrail**: Citation validator checks every cited PMID exists in pubmed_abstracts table. Unverified PMIDs cause brief rejection and agent re-run.

Orchestration

LangGraph state machine with Pydantic-typed state. Conditional edges handle Agent 1 gate (discard below threshold), Agent 2 retry loop, Agent 3 feedback cycle, and HITL routing.

5.6 Guardrails and Human-in-the-Loop (HITL)

Input moderation: RxNorm normalization prevents drug name variants from splitting signal counts. PRR minimum case count (≥ 3) prevents single-report noise from entering the agent pipeline.

Output validation: All LLM outputs validated against Pydantic v2 schemas. Agent 3 citation validator checks every PMID against pubmed_abstracts table before the brief is accepted. Ungrounded claims are rejected and trigger agent re-run.

Schema enforcement: SafetyBrief Pydantic model enforces required fields: severity_level, sss_composite, mechanism_hypothesis, recommendation, confidence_score, supporting_pmids. Missing fields cause structured output rejection.

HITL Gate

Severity	HITL Required	SLA
WEAK	No — auto-logged	None

MODERATE	No — auto-logged	None
HIGH	Yes — reviewer approval required	24 hours
CRITICAL	Yes — reviewer approval required	4 hours

HIGH and CRITICAL signals are written to the hitl_decisions queue. The system cannot publish these signals without a human decision row. All reviewer decisions are immutably logged with reviewer ID, timestamp, and notes.

Confidence thresholding: Agent 3 output with confidence < 0.75 triggers automatic deeper retrieval before a brief is finalized. Briefs that fail to reach 0.75 after 2 retries are held in "pending review" state.

5.7 Evaluations and Testing

MedSignal implements four distinct testing layers covering unit correctness, component integration, LLM output quality, and full system behavior. All tests live under tests/ and are organized by layer.

Layer 1 — Unit Tests

Testing individual functions in complete isolation. No database, no API, no external calls, all dependencies are mocked. Written alongside the code, not after.

Test	What is asserted	Owner
PRR formula	Given A/B/C/D counts, assert PRR = expected. Edge cases: A=0, single case, zero denominator	Samiksha

ROR formula + CI	Given same counts, assert ROR and lower CI bound above 1.0	Samiksha
SSS formula	Given 4 sub-scores, assert composite output. Non-compensatory rule: StatScore=0 must cap at WEAK regardless of other scores	Prachi
Pydantic schema validation	Feed invalid inputs (missing fields, out-of-range confidence, wrong enum), assert validation errors raised	Prachi
RxNorm normalizer	"Ozempic", "ozempic 1mg", "SEMA" → assert all map to same canonical RxCUI	Samiksha
Citation validator	PMIDs list with one fabricated PMID → assert brief is rejected before publication	Prachi
HITL SLA logic	Signal created at T → assert escalation triggers at T+4hr (CRITICAL) and T+24hr (HIGH)	Siddharth
SSS severity tiers	SSS=0.82 → CRITICAL, SSS=0.60 → HIGH, SSS=0.40 → MODERATE, SSS=0.10 → WEAK	Prachi

Tools: pytest, unittest.mock, pytest-mock

Layer 2 — Integration Tests

Testing two or more components working together with real infrastructure. PostgreSQL, Kafka, and Redis run via testcontainers-python. GPT-4o calls are mocked with pre-recorded responses.

Test	Components	Pass condition	Owner
FAERS → Kafka → Spark → PostgreSQL	Kafka producer + Spark + PostgreSQL	After replaying 1,000 FAERS records, drug_symptom_pairs has finasteride-depression PRR within ±0.15 of 3.14	Samiksha
PRR threshold gate	Spark + PostgreSQL + Kafka	Pairs above PRR≥2.0 with count≥3 appear in signals_flagged topic. Pairs below threshold do not appear.	Samiksha
ChromaDB retrieval	ChromaDB + embedding pipeline	Query returns ≥1 abstract with RRF score above threshold, sourced from either DENSE or SPARSE retriever. BOTH source tag considered highest confidence with drug_name=finasteride metadata filter	Samiksha
Agent pipeline (mocked GPT-4o)	Agent 1 + Agent 2 + Agent 3	Given seeded PRR record, full pipeline produces valid SafetyBrief JSON matching Pydantic schema	Prachi
Feedback loop trigger	Agent 3 → Agent 2 (mocked)	When Agent 3 confidence < 0.75, assert Agent 2 is called a second time with n_results=10 not 5	Prachi

HITL gate enforcement	Agent 3 + PostgreSQL + HITL service	CRITICAL signal cannot be written to published signals table without a row in hitl_decisions	Siddharth
FastAPI endpoints	FastAPI + PostgreSQL	GET /signals/active returns correct shape, severity filter, and pagination	Siddharth
Redis cache hit	FastAPI + Redis + PostgreSQL	Second identical request to /signals/active served from Redis assert cache_hit=True in response header	Siddharth

Tools: pytest, pytest-asyncio, httpx, testcontainers-python

Layer 3 — LLM Evaluation Tests

Dedicated eval harness for assessing agent output quality. These tests produce scored metrics reported in Section 9.1 they are not binary pass/fail.

A. Detection Lag Evaluation (primary golden set)

The core research question: does MedSignal detect known historical signals before the FDA's official communication date?

DLS = FDA_communication_date − MedSignal_first_flag_date (days)

Target: Average DLS > 90 days across 10 golden signals

Drug	Signal	FDA Communication
Finasteride	Major Depressive Disorder	2022 label update
Rofecoxib (Vioxx)	Myocardial Infarction	Sep 2004 withdrawal
Ondansetron (Zofran)	QT Prolongation	Sep 2011
Simvastatin	Myopathy / Rhabdomyolysis	Jun 2011 label update
Metformin	Lactic Acidosis	Apr 2016 label update
+ 5 additional	TBD from FAERS literature	Various

B. RAG Retrieval Quality

For 20 drug-symptom pairs with established literature, a team member annotates the 5 most relevant PubMed abstracts as ground truth. Agent 2's retrieval results are scored against this annotation.

Metrics: Mean Reciprocal Rank (MRR), Precision@5, NDCG@5

C. NER Extraction Quality

BioBERT BC5CDR extraction evaluated against n2c2 human-annotated clinical NER benchmarks; gold-standard drug and adverse event entity labels produced by human clinical annotators, not any LLM.

Metrics: Precision, Recall, F1 on drug entity extraction and adverse reaction extraction evaluated separately.

D. Safety Brief Linguistic Quality (secondary)

GPT-4o evaluates generated safety briefs on two narrow dimensions, only fluency and citation format correctness. Not clinical accuracy (evaluating clinical reasoning with GPT-4o would be circular, the same model generating and grading). Scores 1–5 per dimension. Reported as a supplementary metric with the circularity limitation explicitly acknowledged.

Tools: Custom Python eval harness (tests/eval/), GPT-4o API (linguistic eval only)

Layer 4 — End-to-End (E2E) System Tests

Full stack running - Kafka, Spark, all three agents, FastAPI, PostgreSQL, ChromaDB, Redis all live. No mocks. Validates that the system behaves correctly as an integrated whole.

Test	Flow	Pass condition
Demo flow	FAERS record → Kafka → Spark → PRR above threshold → Agent pipeline → SafetyBrief → HITL queue	SafetyBrief appears in /hitl/queue within 60 seconds
HITL approval flow	Reviewer POSTs approve decision → signal moves from pending to published	Signal status = PUBLISHED, hitl_decisions row exists, signal visible in /signals/active
Confidence retry flow	Seed signal where Agent 3 scores low confidence → assert Agent 2 called twice	agent_traces table shows 2 rows for Agent 2 on same signal_id
Cache invalidation	Signal published → Redis TTL expires → next request hits PostgreSQL	Fresh data returned after 60s
PRR checkpoint assertion	Full FAERS 2023 Q1–Q4 batch through Spark	drug_symptom_pairs has finasteride-depression PRR within ±0.15 of 3.14

Tools: pytest, full Docker Compose stack

Test File Structure

```
medsignal/tests/
├── unit/
│   ├── test_prr.py          ← PRR + ROR formula
│   ├── test_sss.py          ← SSS formula + severity tiers
│   ├── test_schemas.py      ← Pydantic validation
│   ├── test_rxnorm.py       ← Drug normalization
│   ├── test_citation.py     ← Citation grounding validator
│   └── test_hitl_sla.py     ← SLA escalation logic
├── integration/
│   ├── test_pipeline.py     ← FAERS → Kafka → Spark → Postgres
│   ├── test_chromadb.py     ← Embedding + retrieval
│   ├── test_agents.py       ← Agent pipeline (mocked GPT-4o)
│   ├── test_fastapi.py      ← API endpoints + Redis cache
│   └── test_hitl_gate.py    ← HITL enforcement
├── e2e/
│   ├── test_demo_flow.py ← Full stack signal → HITL queue
│   └── test_checkpoint.py  ← finasteride PRR ≈ 3.14
└── eval/
        ├── golden_set_eval.py  ← Detection lag (DLS)
        ├── rag_eval.py       ← MRR + Precision@5 + NDCG@5
```

```
└── brief_quality_eval.py ← GPT-4o linguistic eval
```

Testing Ownership and Schedule

Layer	Owner	When
Unit — PRR, ROR, RxNorm, Spark schemas	Samiksha	Written alongside code in W1
Unit — SSS, Pydantic, citation validator	Prachi	Written alongside code in W1–W2
Unit — HITL SLA, FastAPI models	Siddharth	Written alongside code in W2
Integration — pipeline + ChromaDB	Samiksha	W2 D5
Integration — agent pipeline (mocked)	Prachi	W2 D6–D7
Integration — FastAPI + Redis + HITL	Siddharth	W2 D6–D7
E2E tests	All	W3 D3–D4
Golden set eval + RAG eval	Samiksha + Prachi	W3 D2–D4
Brief quality eval (GPT-4o)	Prachi	W3 D4

System Performance Metrics

Metric	Target	Verification Method
Average detection lag (DLS)	< 90 days earlier than FDA	SELECT AVG(fda_communication_date - medsignal_first_flag) FROM golden_signals
Signals detected earlier (of 10)	≥ 7 of 10	SELECT COUNT(*) FROM golden_signals WHERE detection_lag_days > 0
Signal detection precision	> 0.60	Confirmed signals / total flagged signals in test window
False positive rate	< 0.40	1 - precision from above
NER F1 on drug entity extraction	> 0.80	BioBERT output vs n2c2 annotated test set via sklearn.metrics.f1_score
RAG Precision@5	> 0.70	Agent 2 top-5 PMIDs vs pharmacist-annotated ground truth for 20 queries
Mean agent latency (flag to brief)	< 120 seconds	SELECT agent_name, AVG(latency_ms) FROM agent_traces GROUP BY agent_name
Redis cache hit rate	> 40%	SELECT COUNT(*) FILTER (WHERE cache_hit) * 1.0 / COUNT(*) FROM agent_traces

Daily Claude API cost	< $5/day	SELECT DATE(created_at), SUM(cost_usd) FROM agent_traces GROUP BY DATE(created_at)

5.8 Proof of Concept

PRR validation (Week 1 Day 5 checkpoint): After the Spark pipeline processes FAERS 2023 Q1–Q4, a direct query to drug_symptom_pairs for finasteride-depression must return PRR ≈ 3.14. This is the single most important validation in the project it confirms the 7-file join, drug normalization, and PRR computation are all correct. If this fails, the join logic has a cartesian product bug that must be fixed before any agent work begins.

PubMed × ChromaDB hybrid retrieval POC validated: 500-abstract sample produced lit_scores of 0.49–0.66 across 5 golden signal drug-symptom pairs. Zero overlap between HNSW and BM25 retrievers across all 4 test queries confirms hybrid architecture is necessary. warfarin × skin necrosis case demonstrates BM25 catching exact-term matches that HNSW missed entirely at this corpus size.

Agent pipeline POC (Week 2 Day 5): End-to-end run: finasteride-depression signal enters Agent 1 from the database. Agent 2 retrieves 5 PubMed abstracts from ChromaDB on finasteride neurosteroid mechanism. Agent 3 generates a SafetyBrief JSON with inline PMID citations and SSS = 0.82 (CRITICAL). HITL gate fires. Brief appears in the Streamlit review queue.

EDA Findings (pre-implementation)

- FAERS 2023–2024: 16M adverse event records across 8 quarterly ZIPs. Top drugs by report frequency confirm scope : ibuprofen, metformin, finasteride all present with sufficient case counts for PRR computation.
- PubMed corpus: 28,014 abstracts downloaded for 10 golden signal drugs. Average abstract length ~250 words. All records have PMID, title, and abstract text.
- ClinicalTrials.gov API: verified working (HTTP 200). Client code saved at utils/clinicaltrials_client.py. RxNorm API: verified working. Client code saved at utils/rxnorm_client.py.

Project Plan and Timeline

6.1 Milestones

Milestone	Description	End of
Samiksha	Docker Compose stack running. FAERS Kafka producer live.	Week 1 Day 3
Prachi	Spark pipeline complete. PRR ≈ 3.14 for finasteride.	Week 1 Day 5
Siddharth	ChromaDB loaded. All three agents tested in isolation.	Week 2 Day 2
M4	Full LangGraph pipeline end-to-end. FastAPI endpoints live.	Week 2 Day 5
M5	Streamlit dashboard complete. HITL workflow functional.	Week 3 Day 3
M6	Golden set evaluation complete. Detection lag numbers recorded.	Week 3 Day 4
M7	Final polish, documentation, demo rehearsal, submission.	Week 3 Day 7

6.2 Timeline

Task	Owner	Week 1	Week 2	Week 3
Docker Compose — Kafka + Spark setup	Samiksha	D1–D2		
FAERS ZIP parsing + 5-file join	Samiksha	D1–D3		
RxNorm API client	Samiksha	D2		
Kafka FAERS producer	Samiksha	D3–D4		
Spark NER pipeline (BC5CDR)	Samiksha	D4–D5		
Spark PRR/ROR computation	Samiksha	D5		
PostgreSQL schema — 9 tables	Prachi	D1–D2		
Pydantic shared schemas — all agents	Prachi + All	D2		
PubMed abstract fetch — 28K abstracts	Prachi	D2–D4		
ClinicalTrials API client	Prachi	D3–D4		

Golden signal table — 10 signals + FDA dates	Prachi	D4–D5		
SSS scoring service	Prachi	D5		
Docker Compose — full stack	Siddharth	D1–D2		
Redis — Bloom filter + hot signal cache	Siddharth	D2–D3		
Mock data generator — fake PRR rows	Siddharth	D3–D4		
Airflow DAG — PubMed embedding refresh	Siddharth	D4–D5		
✓ Checkpoint: PRR ≈ 3.14 verified	All	D5		
Airflow DAG — FAERS batch load	Samiksha	—	D1–D2	
Kafka Structured Streaming consumer	Samiksha	—	D2–D4	
ChromaDB embedding load	Samiksha	—	D4–D5	
Agent 1 — signal detector	Samiksha	—	D1–D2	

Agent 2 — literature retriever (RAG)	Prachi	—	D1–D3	
Agent 3 — severity assessor + SSS + feedback loop	Prachi	—	D4–D6	
LangGraph orchestrator — all three wire their node	All	—	D7	
HITL service — review queue + SLA routing	Siddharth	—	D1–D3	
Agent 4 — reviewer briefing (HITL Claude call)	Siddharth	—	D3–D4	
FastAPI — /hitl + /health + /ingest endpoints	Siddharth	—	D3–D4	
FastAPI — /signals + /drugs endpoints	Prachi	—	D5	
FastAPI — /analytics endpoints	Samiksha	—	D5	
Kafka mock producer (frontend testing)	Siddharth	—	D4	
✓ Checkpoint: full pipeline end-to-end	All	—	D7	

Golden set evaluation run	Samiksha	—	—	D2–D4
DLS + precision + F1 metrics	Samiksha	—	—	D4–D5
RAG Precision@5 evaluation	Prachi	—	—	D2–D3
Brief quality evaluation (Claude)	Prachi	—	—	D4
Redis cache layer — agent TTL	Prachi	—	—	D1
React — Evaluation page	Samiksha	—	—	D5
React — System Health page	Prachi	—	—	D5
React — Signal feed + Signal detail	Siddharth	—	—	D1–D2
React — HITL review interface	Siddharth	—	—	D3
React — Drug search page	Siddharth	—	—	D4
Unit + integration tests	Siddharth	—	—	D4–D5

Demo rehearsal + final polish + README	All	—	—	D5–D7

Team Roles and Responsibilities

Member	Role	Primary Ownership	LLM Work
Samiksha Rajesh Gupta	Data Pipeline + Statistical Intelligence Lead	Kafka + Spark + FAERS + PRR/ROR + SSS scoring	Agent 1 : Signal Detector
Prachi Ganpatrao Pradhan	LLM Agent + RAG Lead	LangGraph + Agent 2 + RAG pipeline	Agent 2 : Literature Retriever, LangGraph orchestrator (lead)
Siddharth Rakesh Shukla	Backend + Frontend + Validation Lead	FastAPI + React + HITL + Infrastructure	Agent 3 : Severity Assessor + feedback loop trigger

Risks and Mitigation

8.1 Potential Risks

Risk	Likelihood	Impact

FAERS cartesian product bug in 7-file join produces wrong PRR silently	High	Critical : poisons all downstream signals
Week 1 timeline slip cascades into agent pipeline work	High	High : zero slack in 3-week schedule
Reddit API rate limits or access restrictions	High	Low : optional component with graceful fallback
LLM hallucinated citations undermine safety brief credibility	Medium	High
ChromaDB returns irrelevant papers for a drug-symptom pair	Medium	Medium
OpenAI API cost exceeds $50 budget	Medium	Medium
Team member blocking during integration	Medium	Medium

8.2 Mitigation Strategies

Risk	Mitigation
Cartesian product bug	Filter DRUG file to suspect drugs only (drugcharacterization = 1) before joining. Assert finasteride PRR ≈ 3.14 as hard pipeline test.

Week 1 slip	Hard checkpoint Day 5. If behind: drop Reddit (set PatientScore = 0), defer Airflow (use manual triggers), defer AWS. Never cut Spark or agents.
Reddit API	Build last. PatientScore = 0 fallback in SSS formula. System accurate on 3 sources.
Hallucinated citations	Agent 3 only receives PMIDs passed explicitly by Agent 2. Post-generation validator checks every PMID against pubmed_abstracts. Brief rejection and re-run if validation fails.
Irrelevant ChromaDB results	Always filter by drug_name metadata before cosine search. Minimum similarity threshold 0.75.Additional mitigation: hybrid BM25 + HNSW retrieval ensures exact drug/symptom token matches are always captured even when semantic embedding space is sparse for new or rare drugs.
API cost overrun	Hard $50 limit set in OpenAI dashboard on Day 1. Use GPT-4o-mini during development. Cache all agent responses in Redis.
Integration blocking	Mock-first development. Prachi builds agents against mock drug_symptom_pairs. Siddharth builds Streamlit against mock FastAPI. Real integration in the final 2 days of each week.

Expected Outcomes and Metrics

9.1 KPIs

KPI	Target

Average detection lag over FDA communication dates	> 90 days earlier
Golden signals detected earlier (of 10)	≥ 7 of 10
Signal detection precision	> 0.75
NER F1 on drug entity extraction	> 0.80
RAG Precision@5	> 0.70
Mean agent pipeline latency	< 60 seconds
Redis cache hit rate	> 40%
Daily GPT-4o cost during demo week	< $5/day

9.2 Expected Benefits

MedSignal demonstrates that the data infrastructure required to dramatically reduce pharmacovigilance detection lag already exists; it simply requires the right engineering to unlock. By combining streaming data processing at scale with RAG-grounded LLM reasoning and structured human oversight, the platform reduces what currently takes a human analyst hours of manual work to minutes of automated, citation-backed investigation.

The detection lag evaluation against 10 historical FDA signals provides a concrete, defensible measure of impact. Even a 90-day average lead time over official FDA communications represents a meaningful reduction in patient exposure to unrecognized drug risks.

Token and Cost Report

Cost Drivers

- GPT-4o agent calls (primary cost): 3 agents × average 2,000 tokens per signal × estimated 500 signals processed during demo week = ~3M tokens ≈ $15–25
- OpenAI embedding generation (one-time): 28,000 abstracts × ~330 tokens average = ~9.2M tokens ≈ $0.18
- GPT-4o-mini during development: ~$5 estimated

Hard limits: OpenAI account hard limit set at $50 on Day 1. Budget alert at $30.

Optimization Strategies

- Redis caches all agent responses for identical drug-symptom inputs (24-hour TTL). Target > 40% cache hit rate during demo week reduces GPT-4o calls by nearly half.
- GPT-4o-mini used for all development and testing. GPT-4o used only for final evaluation and demo.
- RxNorm lookups cached in Redis' same drug name variant never hits the API twice.
- Embedding generation is a one-time cost. ChromaDB persists to disk across container restarts.

Token reporting: The agent_traces PostgreSQL table logs every LLM call with total_tokens, cost_usd, latency_ms, and cache_hit. The React System Health page displays cumulative daily token usage and cost by agent in real time via a /analytics/token-cost FastAPI endpoint.

Conclusion

MedSignal addresses a documented, consequential gap in public health infrastructure: the 6–24 month detection lag between when drug safety signals first appear in FDA FAERS data and when patients are officially warned. The platform combines streaming data engineering at scale with a three-agent LLM pipeline grounded in peer-reviewed medical literature, evaluated against a rigorous retrospective golden set of 10 historical FDA signals.

The system is honest about what it is and what it is not. MedSignal is a detection and triage tool it accelerates human review, it does not replace it. The HITL gate exists because LLM-generated safety briefs require clinical judgment before publication. PRR statistical significance is not causation. These constraints are built into the architecture, not acknowledged as footnotes.

What the system does deliver is concrete: a platform that reads 16 million adverse event reports, retrieves supporting medical evidence, and produces a structured safety brief with full citation trails in minutes rather than months. That is the gap MedSignal closes.

References

1. FDA Adverse Event Reporting System (FAERS). https://fis.fda.gov/extensions/FPD-QDE-FAERS/FPD-QDE-FAERS.html
2. openFDA Drug Adverse Event API. https://open.fda.gov/apis/drug/event/
3. NCBI E-utilities API. https://eutils.ncbi.nlm.nih.gov/
4. ClinicalTrials.gov REST API v2. https://clinicaltrials.gov/api/v2
5. RxNorm API. https://rxnav.nlm.nih.gov/
6. Freifeld, C.C., et al. (2014). Digital Drug Safety Surveillance. Drug Safety, 37(5), 343–350.
7. Nikfarjam, A., et al. (2015). Pharmacovigilance from social media. JAMIA, 22(3), 671–681.
8. Evans, S.J.W., et al. (2001). Use of proportional reporting ratios for signal generation. Pharmacoepidemiology and Drug Safety, 10(6), 483–486.
9. n2c2 NLP Research Datasets. https://n2c2.dbmi.hms.harvard.edu/
10. LangGraph Documentation. https://langchain-ai.github.io/langgraph/
11. ChromaDB Documentation. https://docs.trychroma.com/

Appendix

A. Core Pydantic Schemas

```
class SeverityLevel(str, Enum):
    WEAK     = "WEAK"
    MODERATE = "MODERATE"
    HIGH     = "HIGH"
    CRITICAL = "CRITICAL"

class SSSScores(BaseModel):
    stat_score:      Decimal  # 0.0 - 1.0
    lit_score:       Decimal
    patient_score: Decimal
    trial_score:   Decimal
    sss_composite: Decimal
    severity_level: SeverityLevel

class SafetyBrief(BaseModel):
    signal_id:       str
    drug_name:               str
    meddra_reaction:         str
    severity_level:          SeverityLevel
    sss_scores:              SSSScores
    mechanism_hypothesis: str
    recommendation:          str     #   MONITOR   /   LABEL_UPDATE   /   RESTRICT   /
WITHDRAW
```

```
confidence_score:     Decimal
supporting_pmids:     list[str]
generated_summary:    str
hitl_required:        bool
```

B. Kafka Topic Schema (FAERS case message)


```
{
 "primaryid":  "12345678",
 "receive_date": "20230415",
 "patient_age":  54,
 "patient_sex":  "M",
 "country":     "US",
 "drugs":       [{ "name": "FINASTERIDE", "rxcui": "184305", "role": "PS" }],
 "reactions": [{ "meddra_term": "Depression", "meddra_code": "10012378" }],
 "outcomes":  ["HO"],
 "indication": "Benign prostatic hyperplasia"
}
```


C. PRR Validation Query

```
SELECT drug_name, meddra_reaction, prr_score, case_count_a
FROM   drug_symptom_pairs
WHERE  drug_name = 'finasteride'
  AND  meddra_reaction ILIKE '%depress%'
ORDER  BY prr_score DESC
LIMIT  5;


-- Expected: prr_score ≈ 3.14,  case_count_a ≈ 678
```

D. Environment Variables

```
OPENAI_API_KEY=...
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
POSTGRES_URL=postgresql://medsignal:password@localhost:5432/medsignal
CHROMA_PATH=./data/chromadb
REDIS_URL=redis://localhost:6379
NCBI_API_KEY=...
```

E. Sample Prompts

Each prompt below is the exact text sent to LLM for the corresponding agent or guardrail step. All prompts use structured output mode — responses are validated against Pydantic schemas before the system accepts them. Variable placeholders are shown in {curly_braces}.

E.1 System Prompts — Agent Identities

Each agent receives a system prompt once when the LangGraph node initialises. These define the agent's role, constraints, and output contract.

E.1.1 Agent 1 — Signal Detector

System prompt:

```
You are Agent 1 — Signal Detector for MedSignal, a pharmacovigilance platform.
Your sole job is to evaluate whether a flagged drug-symptom pair from FDA FAERS
represents a statistically credible adverse drug signal worth investigating further.

You have access to two MCP tools:
  • get_signals: retrieves PRR/ROR scores from the signals_flagged database
  • query_clinicaltrials: fetches adverse event data from ClinicalTrials.gov

Rules you must follow without exception:
  1. Never make claims about biological mechanism. That is Agent 3's job.
  2. Never cite papers. That is Agent 2's job.
  3. If PRR < 2.0 or case count < 3, output signal_strength = WEAK and stop.
  4. Generate exactly 3 PubMed search queries — one each for mechanism,
       epidemiology, and clinical outcomes.
  5. Output must be valid JSON matching the SignalDetectorOutput schema.
       Do not include any text outside the JSON object.
```

E.1.2 Agent 2 — Literature Retriever

System prompt:

You are Agent 2 — Literature Retriever for MedSignal, a pharmacovigilance platform.

Your sole job is to find the most relevant peer-reviewed PubMed abstracts for a

given drug-symptom signal, using semantic search over a ChromaDB vector store.

You have access to two MCP tools:

- search_pubmed: queries ChromaDB using cosine similarity (384-dim MiniLM-L6-v2)
- query_chroma: retrieves abstracts by PMID for verification

Rules you must follow without exception:

1. Always filter by drug_name metadata BEFORE running cosine similarity search.
2. Only return abstracts with cosine similarity ≥ 0.75.
3. If best similarity < 0.75, broaden the query to the MedDRA parent term

   and retry. Maximum 3 attempts before returning best available.
4. Return exactly 5 abstracts unless n_results override is specified.
5. Every PMID you return must exist in the pubmed_abstracts table.

   Do not fabricate or paraphrase PMIDs.
6. Output must be valid JSON matching the LitRetrieverOutput schema.

E.1.3 Agent 3 — Severity Assessor

System prompt:

You are Agent 3 — Severity Assessor for MedSignal, a pharmacovigilance platform.

Your job is to synthesise statistical signal data, peer-reviewed literature, and

clinical trial adverse event records into a structured SafetyBrief.

You have access to two MCP tools:

- get_safety_brief: retrieves prior briefs for the same drug-symptom pair
- flag_signal: writes the completed SafetyBrief to the signals_flagged topic

Rules you must follow without exception:

1. Use ONLY the PMIDs provided in the literature input. Never cite any PMID

   not present in supporting_abstracts. Fabricated citations cause brief rejection.

2. Compute SSS using this formula:

   SSS = 0.40 × StatScore + 0.30 × LitScore + 0.20 × PatientScore + 0.10 × TrialScore

3. If StatScore = 0, set severity_level = WEAK regardless of other scores.

4. Set confidence_score between 0.0 and 1.0. If confidence < 0.75, the system

   will trigger a retry — request deeper literature retrieval in your reasoning.

5. recommendation must be one of: MONITOR, LABEL_UPDATE, RESTRICT, WITHDRAW.

6. Output must be valid JSON matching the SafetyBrief Pydantic schema.

   Do not include any text outside the JSON object.

E.2 User Prompts — Per-Signal Invocations

These prompts are constructed dynamically for each signal and passed as the user message. Each one is built by the LangGraph orchestrator by injecting values from the pipeline state.

E.2.1 Agent 1 User Prompt — Signal Evaluation

User prompt (constructed per signal):

Evaluate the following drug-symptom signal from FDA FAERS and determine whether it warrants further investigation.

Signal data:

drug_name: {drug_name}

rxcui: {rxcui}

meddra_term: {meddra_term}

meddra_code: {meddra_code}

prr_score: {prr_score}

ror_score: {ror_score}

case_count: {case_count}

window_start: {window_start}

window_end: {window_end}

death_count: {death_count}

hosp_count: {hosp_count}

Steps:

1. Call get_signals to retrieve full signal context from the database.
2. Call query_clinicaltrials to check adverse event corroboration.
3. Evaluate statistical strength. PRR >= 2.0 and case_count >= 3 are required.
4. Generate 3 PubMed search queries:

    query_1: biological mechanism between {drug_name} and {meddra_term}

```
    query_2: epidemiology and incidence of {meddra_term} in {drug_name} users
    query_3: clinical outcomes and risk factors for {meddra_term} in {drug_name}
5. Return JSON matching SignalDetectorOutput. Do not add prose outside the JSON.
```

E.2.2 Agent 2 User Prompt — Literature Retrieval

User prompt (constructed per signal):

Retrieve the most relevant PubMed abstracts for this drug-symptom signal.

Signal context:

drug_name: {drug_name}

meddra_term: {meddra_term}

signal_id: {signal_id}

Search queries from Agent 1:

query_1: {evidence_queries[0]}

query_2: {evidence_queries[1]}

query_3: {evidence_queries[2]}

Retrieval instructions:

1. For each query, call search_pubmed with drug_name filter = {drug_name}.
2. Accept only abstracts with cosine_similarity >= 0.75.
3. If similarity < 0.75 on all three queries, broaden to MedDRA parent term:
   {meddra_parent_term} and retry once.
4. Return the top 5 unique abstracts ranked by relevance score.
5. For each abstract, include: pmid, title, excerpt (first 200 chars),
   cosine_similarity, pub_year, journal.
6. Return JSON matching LitRetrieverOutput. No prose outside the JSON.

E.2.3 Agent 3 User Prompt — Safety Brief Generation

User prompt (constructed per signal):

Generate a structured SafetyBrief for the following drug-symptom signal.

Synthesise all evidence streams into a single clinical assessment.

Statistical evidence (from Agent 1):

drug_name: {drug_name}

meddra_term: {meddra_term}

prr_score: {prr_score}

ror_score: {ror_score}

case_count: {case_count}

death_count: {death_count}

hosp_count: {hosp_count}

stat_score: {stat_score}

trial_score: {trial_score}

patient_score: {patient_score}

Literature evidence (from Agent 2):

{supporting_abstracts}

[Each abstract includes: pmid, title, excerpt, cosine_similarity]

Instructions:

1. Compute lit_score based on abstract relevance (average cosine_similarity of top-5 abstracts, normalised to 0.0-1.0).

2. Compute sss_composite = 0.40*stat_score + 0.30*lit_score

+ 0.20*patient_score + 0.10*trial_score.

3. Assign severity_level:

WEAK if sss_composite <= 0.25 or stat_score = 0

MODERATE if 0.26 <= sss_composite <= 0.50

HIGH if 0.51 <= sss_composite <= 0.75

CRITICAL if sss_composite > 0.75

4. Write mechanism_hypothesis in 2-3 sentences. Cite only PMIDs from the

supporting_abstracts list. Format: (PMID: 12345678).

5. Write recommendation: one of MONITOR / LABEL_UPDATE / RESTRICT / WITHDRAW.

6. Set confidence_score. If < 0.75, the system will request deeper retrieval.

7. Return JSON matching SafetyBrief schema. No prose outside the JSON.

E.3 Confidence Retry Prompt — Agent 3 → Agent 2

When Agent 3 returns confidence < 0.75, the LangGraph conditional edge fires this prompt back to Agent 2, requesting deeper retrieval before a final brief is produced. Maximum 2 retries.

Retry prompt:

```
Agent 3 evaluated the SafetyBrief for {drug_name} — {meddra_term} and returned
confidence_score = {confidence_score}, which is below the 0.75 threshold.

Agent 3's low-confidence reasoning: {agent3_reasoning}

Retry instructions (attempt {retry_count} of 2):
  1. Increase n_results from 5 to 10.
  2. Add this query specifically targeting Agent 3's uncertainty:
        {retry_query}
  3. Accept abstracts with cosine_similarity >= 0.65 on this retry only.
  4. Return all results in LitRetrieverOutput format.

If no additional relevant abstracts are found, return what you have.
Agent 3 will make a final determination regardless.
```

E.4 Citation Grounding Validator

This is not an LLM prompt — it is a deterministic post-generation check run before Agent 3's output is accepted. Any PMID in the SafetyBrief that does not exist in the pubmed_abstracts table causes immediate rejection and re-run.

Citation validator logic (Python):

```
def validate_citations(brief: SafetyBrief, db: Session) -> ValidationResult:
    for pmid in brief.supporting_pmids:
    exists = db.query(PubmedAbstract).filter_by(pmid=pmid).first()
    if not exists:
    raise CitationGroundingError(
        f'PMID {pmid} not found in pubmed_abstracts. '
        f'Brief rejected. Agent 3 will re-run.'
    )
    return ValidationResult(status='PASSED', pmid_count=len(brief.supporting_pmids))
```

E.5 HITL Reviewer Briefing Prompt

When a HIGH or CRITICAL signal enters the HITL queue, the system generates a plain-language reviewer briefing from the SafetyBrief JSON. This is presented to the pharmacist reviewer in the React HITL interface alongside the approve / reject / modify controls.

Reviewer briefing prompt:

You are preparing a clinical review briefing for a pharmacovigilance analyst. Convert the following structured SafetyBrief into a plain-language summary suitable for a pharmacist or physician reviewer. Be factual, concise, and do not add speculation beyond what the data supports.

SafetyBrief data:

drug_name: {drug_name}
meddra_term: {meddra_term}
severity_level: {severity_level}
sss_composite: {sss_composite}
prr_score: {prr_score}
case_count: {case_count}
death_count: {death_count}
hosp_count: {hosp_count}
mechanism_hypothesis: {mechanism_hypothesis}
recommendation: {recommendation}
confidence_score: {confidence_score}
supporting_pmids: {supporting_pmids}

Write:

1. A one-sentence signal summary (drug + reaction + severity).
2. Why this is statistically notable (PRR, case count, outcomes).

3. What the literature says (2-3 sentences, cite PMIDs inline).

4. The system recommendation and what action that implies.

5. Any uncertainty or limitations the reviewer should know.

Format as plain prose. No JSON. No headers. Maximum 200 words.

The reviewer will approve, reject, or escalate based on your summary.